# 1 Primes, Primality Testing, Factorization, CRT and Discrete Logs

Asymmetric-key cryptography is built on a handful of facts about primes, and this chapter collects them. RSA needs two large random primes, so we need a fast way to decide whether a 1024-bit number is prime; this is *primality testing*. RSA is secure only because nobody can split $n = pq$ back into $p$ and $q$; this is the hardness of *factorization*. RSA decryption is made about four times faster by working modulo $p$ and $q$ separately and gluing the results with the *Chinese remainder theorem*. Every public-key operation is a modular power, computed by *square-and-multiply*. Finally, Diffie–Hellman and ElGamal rely on the fact that undoing a modular power, the *discrete logarithm*, is infeasible.

## 1.1 Primes and how many there are

A positive integer is prime if it has exactly two divisors, 1 and itself; a *composite* has more than two divisors, and the number 1 (exactly one divisor) is neither. Two integers are *coprime* (relatively prime) when their gcd is 1, so a prime $p$ is coprime to every integer from 1 to $p-1$, which is why $\mathbb{Z}_p^* = \{1, \dots, p-1\}$.

**Infinitely many primes.** Suppose the primes were only $p_1, \dots, p_k$, and let $N = p_1 p_2 \cdots p_k + 1$. Dividing $N$ by any $p_i$ leaves remainder 1, so $N$ either is a new prime or has a prime factor outside the list; either way the list was incomplete. Note that $N$ itself need not be prime: $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 + 1 = 30031 = 59 \times 509$, but 59 and 509 are new primes.

**Counting primes.** The function $\pi(n)$ counts the primes $\le n$: $\pi(1) = 0$, $\pi(2) = 1$, $\pi(3) = 2$, $\pi(10) = 4$, $\pi(20) = 8$, $\pi(50) = 15$, $\pi(100) = 25$. For large $n$ there is no formula, only bounds (Lagrange found the lower one, Gauss the upper):

$$\frac{n}{\ln n} < \pi(n) < \frac{n}{\ln n - 1.08366}.$$

For $n = 10^6$: $n/\ln n = 72{,}382.4$ and $n/(\ln n - 1.08366) = 78{,}543.2$, so $72{,}383 \le \pi(10^6) \le 78{,}543$; the actual count is 78,498. The practical consequence is that primes thin out only slowly: near $N$ about one number in $\ln N$ is prime. For 1024-bit numbers $\ln 2^{1024} \approx 710$, so if we test random *odd* 1024-bit numbers, about one in 355 is prime, and generating an RSA prime takes a few hundred primality tests. This is why the test itself must be fast.

**Checking a number by $\sqrt{n}$.** If $n = ab$ with $1 < a \le b$, then $a^2 \le ab = n$, so $a \le \sqrt{n}$. Therefore a composite $n$ always has a prime factor $\le \lfloor \sqrt{n} \rfloor$, and it is enough to try the primes up to $\lfloor \sqrt{n} \rfloor$.

- Is 97 prime? $\lfloor \sqrt{97} \rfloor = 9$, and none of 2, 3, 5, 7 divides 97, so **97 is prime**.
- Is 301 prime? $\lfloor \sqrt{301} \rfloor = 17$, so we would try 2, 3, 5, 7, 11, 13, 17; but already $7 \mid 301$ ($301 = 7 \times 43$), so **301 is composite**.

**Mersenne and Fermat numbers.** These were early attempts at formulas that produce primes. A *Mersenne number* is $M_p = 2^p - 1$. If the exponent factors as $ab$, then $2^a - 1$ divides $2^{ab} - 1$ (since $x - 1 \mid x^b - 1$ with $x = 2^a$), so $M_p$ can be prime only for prime $p$. The condition is not sufficient: $M_2, M_3, M_5, M_7, M_{13}, M_{17}, M_{19} = 3, 7, 31, 127, 8191, 131071, 524287$ are prime, but $M_{11} = 2047 = 23 \times 89$ and $M_{23} = 8388607 = 47 \times 178481$. A *Fermat number* is $F_n = 2^{2^n} + 1$. Fermat saw that $F_0, \dots, F_4 = 3, 5, 17, 257, 65537$ are prime and guessed that all are, but Euler found $F_5 = 4294967297 = 641 \times 6700417$, and no larger Fermat number is known to be prime. $F_4 = 65537 = 2^{16} + 1$ survives in practice as the standard RSA public exponent, because its binary form has only two 1-bits, which makes encryption with square-and-multiply cheap.

## 1.2 Primality testing: deterministic and probabilistic

The problem is to decide, for a number of several hundred digits, whether it is prime. A *deterministic* algorithm always gives the correct answer; a *probabilistic* algorithm is much faster but may declare a composite number "prime" with a small, controllable probability. Its "composite" answers are always correct, because they come with a witness.

**Divisibility test (deterministic).** Try every $r = 2, 3, \dots, \lfloor \sqrt{n} \rfloor$ and return "composite" as soon as $r \mid n$, otherwise "prime". It is correct by the $\sqrt{n}$ argument above, but its cost is exponential in the size of the input: an $n_b$-bit number has $\sqrt{n} \approx 2^{n_b/2}$ candidate divisors. For a 200-bit $n$ this is $2^{100}$ bit operations; at $2^{30}$ operations per second that takes $2^{70}$ seconds, about $3.7 \times 10^{13}$ years.

**AKS (deterministic, polynomial).** In 2002 Agrawal, Kayal and Saxena gave the first deterministic test whose time is polynomial in the number of bits, $O((\log_2 n)^{12})$ bit operations, based on the identity $(x - a)^p \equiv x^p - a \pmod p$, which holds exactly when $p$ is prime. It settled a theoretical question, but in practice the probabilistic tests below are far faster.

**Fermat test (probabilistic).** Fermat's little theorem says that if $n$ is prime then $a^{n-1} \equiv 1 \pmod n$ for every $a$ with $n \nmid a$. Read in reverse: if $a^{n-1} \not\equiv 1 \pmod n$, then $n$ is certainly composite; if $a^{n-1} \equiv 1$, then $n$ is only *probably* prime, because some composites also satisfy it. For example $341 = 11 \times 31$ gives $2^{340} \equiv 1 \pmod{341}$ (since $2^{10} = 1024 = 3 \cdot 341 + 1$), so base 2 is fooled, but base 3 exposes it: $3^{340} \equiv 56 \pmod{341}$.

**Carmichael numbers.** Trying more bases does not always help. A *Carmichael number* is a composite $n$ with

$a^{n-1} \equiv 1 \pmod{n}$ for *every* $a$ coprime to $n$, so the Fermat test fails for all such bases. The smallest is $561 = 3\times11\times17$ (then 1105, 1729, 2465, 2821). The reason is visible from the factors: $561 - 1 = 560$ is divisible by $3 - 1$, $11 - 1$ and $17 - 1$, so for each prime factor $p$, $a^{560} = (a^{p-1})^{560/(p-1)} \equiv 1 \pmod{p}$, and a number that is $\equiv 1$ modulo 3, 11 and 17 is $\equiv 1$ modulo 561 (by the CRT below). In general $n$ is Carmichael exactly when it is squarefree and $(p-1) \mid (n-1)$ for every prime $p \mid n$ (Korselt's criterion). Only bases sharing a factor with $n$ catch it, e.g. $3^{560} \equiv 375 \pmod{561}$, and for large $n$ such bases are too rare to find by chance.

**Square-root test.** If $n = p$ is prime and $x^2 \equiv 1 \pmod{p}$, then $p \mid (x-1)(x+1)$, and since $p$ is prime it divides one of the two factors, so $x \equiv \pm1 \pmod{p}$. Hence modulo a prime, the only square roots of 1 are $+1$ and $-1$ (written $n-1$ for $-1$). A composite modulus usually has extra, *nontrivial* square roots of 1, and finding one proves that $n$ is composite.

| $n$ | Square roots of 1 mod $n$ | Comment |
|---|---|---|
| 7 (prime) | 1, 6 | only $\pm1$ |
| 17 (prime) | 1, 16 | only $\pm1$ |
| 8 (composite) | 1, 3, 5, 7 | extra roots 3, 5 $\Rightarrow$ composite |
| 15 (composite) | 1, 4, 11, 14 | extra roots 4, 11 |
| 22 (composite) | 1, 21 | only $\pm1$ although composite |
| 561 (composite) | 1, 67, 188, 254, 307, 373, 494, 560 | $2^3 = 8$ roots (3 prime factors) |

The test is one-way: $n = 22 = 2 \times 11$ has only $\pm1$, because modulo 2 there is a single square root of 1. For an odd $n$ with $r$ distinct prime factors there are $2^r$ square roots of 1 (a choice of $\pm1$ modulo each prime factor, glued by the CRT). A nontrivial root also *factors* $n$: since $n \mid (x-1)(x+1)$ but divides neither factor, $\gcd(x \pm 1, n)$ are proper factors; for $x = 67$, $\gcd(66, 561) = 33$ and $\gcd(68, 561) = 17$.

### 1.3 Miller–Rabin test ▸

▸ **Likely this year:** "Use the Miller–Rabin test with base 2 to check whether 561 (or 61, 341) is prime": write $n - 1 = m \cdot 2^k$, then the $T$ table.

The Fermat test only looks at the final value $a^{n-1}$, and Carmichael numbers make that value 1 for every base. Miller–Rabin also looks at the values *on the way* to $a^{n-1}$ and applies the square-root test to them, which catches Carmichael numbers too.

**Idea.** Write $n - 1 = m \cdot 2^k$ with $m$ odd ($n$ odd, so $k \geq 1$). Then $a^{n-1}$ is obtained from $a^m$ by squaring $k$ times:

$$a^m,\ a^{2m},\ a^{4m},\ \ldots,\ a^{2^k m} = a^{n-1} \pmod{n},$$

each term being the square of the previous one. If $n$ is prime, the last term is 1 by Fermat. Walking backwards from it, every term whose square is 1 must be $\pm1$, by the square-root test. So for a prime $n$ the sequence either starts with $a^m \equiv 1$ (then it is all 1s), or it contains a $-1$ somewhere before the last position, after which it is all 1s. If the sequence reaches $+1$ *without* a $-1$ just before it, the previous term is a nontrivial square root of 1 and $n$ is composite; if it never reaches $\pm1$, Fermat already fails.

**★ MUST KNOW : Miller–Rabin (base $a$)**

Find $m$ odd and $k$ with $n - 1 = m \times 2^k$. Set $T \leftarrow a^m \bmod n$. If $T = \pm1$, return **prime**.
Repeat at most $k - 1$ times: $T \leftarrow T^2 \bmod n$; if $T = +1$ return **composite** (the previous $T$ was a nontrivial square root of 1); if $T = -1$ return **prime**.
If the loop ends without a decision, return **composite**.

The loop stops after $k - 1$ squarings because the $k$-th square is $a^{n-1}$ itself: if the previous $T$ is not $\pm1$, then either its square is 1 (a nontrivial root) or it is not 1 (Fermat fails), so $n$ is composite in both cases and nothing is gained by computing it. "Prime" here means "probably prime": a composite $n$ passes for at most $1/4$ of the bases (such bases are called *strong liars*), so $t$ independent random bases leave an error probability of at most $4^{-t}$, e.g. $4^{-40} \approx 10^{-24}$. The *recommended primality test* (Forouzan) is therefore trial division by small primes first, which cheaply removes most composites, followed by Miller–Rabin with several bases.

**WORKED EXAMPLE : Miller–Rabin on 561 and 61 (Forouzan, base 2)**

**Q.** Use the Miller–Rabin test with base 2 to decide whether (a) 561 and (b) 61 are prime.
**(a)** $n - 1 = 560 = 35 \times 2^4$, so $m = 35$, $k = 4$, $a = 2$, and at most $k - 1 = 3$ squarings. First $2^{35} \bmod 561$ by repeated squaring: $2^8 = 256$, $2^{16} = 256^2 \bmod 561 = 460$, $2^{32} = 460^2 \bmod 561 = 103$, so $2^{35} = 2^{32} \cdot 2^3 \equiv 103 \times 8 = 824 \equiv 263$.

| Step | Computation | $T$ | Decision |
|---|---|---|---|
| Init | $2^{35} \bmod 561$ | 263 | not ±1, continue |
| 1 | $263^2 = 69169 \bmod 561$ | 166 | not ±1, continue |
| 2 | $166^2 = 27556 \bmod 561$ | 67 | not ±1, continue |
| 3 | $67^2 = 4489 \bmod 561$ | **1** | +1 ⇒ **composite** |

**561 is composite.** Justification: $67^2 \equiv 1$ with $67 \neq \pm 1$, so 67 is a nontrivial square root of 1, impossible modulo a prime. (Indeed $561 = 3 \times 11 \times 17$; and $\gcd(66, 561) = 33$, $\gcd(68, 561) = 17$.) Note that one more squaring gives $2^{560} \equiv 1$, so 561 *passes* the Fermat test; Miller–Rabin catches it only because it inspects the step from 67 to 1.

**(b)** $n-1 = 60 = 15 \times 2^2$, so $m = 15$, $k = 2$, at most one squaring. $2^{15} = 2^8 \cdot 2^4 \cdot 2^2 \cdot 2 \equiv 12 \times 16 \times 4 \times 2 = 1536 = 25 \cdot 61 + 11 \equiv 11 \pmod{61}$.

| Step | Computation | $T$ | Decision |
|---|---|---|---|
| Init | $2^{15} \bmod 61$ | 11 | not ±1, continue |
| 1 | $11^2 = 121 \bmod 61$ | $60 = -\mathbf{1}$ | $-1$ ⇒ **prime** |

**61 passes (probably prime)**, and 61 is indeed prime. Check: $2^{60} = (2^{30})^2 \equiv (-1)^2 = 1$, consistent with Fermat.

**WORKED EXAMPLE : Fermat test vs Miller–Rabin on 341 (new)**

**Q.** Show that 341 passes the Fermat test with base 2, then apply the Miller–Rabin test with base 2. What can you conclude?
**Fermat test.** $2^{10} = 1024 = 3 \times 341 + 1$, so $2^{10} \equiv 1 \pmod{341}$ and $2^{340} = (2^{10})^{34} \equiv 1$. 341 passes the Fermat test with base 2.
**Miller–Rabin.** $340 = 85 \times 2^2$, so $m = 85$, $k = 2$, at most one squaring. $2^{85} = (2^{10})^8 \cdot 2^5 \equiv 1 \cdot 32 = 32$.

| Step | Computation | $T$ | Decision |
|---|---|---|---|
| Init | $2^{85} \bmod 341$ | 32 | not ±1, continue |
| 1 | $32^2 = 1024 \bmod 341$ | **1** | +1 ⇒ **composite** |

**341 is composite**: 32 is a nontrivial square root of 1 modulo 341. Base 2 is a Fermat liar for 341 but not a strong liar. The root even gives the factors: $\gcd(31, 341) = 31$ and $\gcd(33, 341) = 11$, so $341 = 11 \times 31$.

**WORKED EXAMPLE : a prime with a long sequence (new)**

**Q.** Apply the Miller–Rabin test with base 2 to $n = 97$.
$96 = 3 \times 2^5$, so $m = 3$, $k = 5$, at most 4 squarings.

| Step | Computation | $T$ | Decision |
|---|---|---|---|
| Init | $2^3 \bmod 97$ | 8 | continue |
| 1 | $8^2 = 64$ | 64 | continue |
| 2 | $64^2 = 4096 = 42 \cdot 97 + 22$ | 22 | continue |
| 3 | $22^2 = 484 = 5 \cdot 97 - 1$ | $96 = -\mathbf{1}$ | **prime** |

**97 passes** (and is prime, as the $\sqrt{97}$ check showed earlier).

The remaining slide runs and two instructive edge cases:

| $n$, base | $n - 1 = m \cdot 2^k$ | $T$ sequence | Result |
|---|---|---|---|
| 27, 2 | $13 \cdot 2^1$ | 11 (no loop, $k - 1 = 0$) | composite (correct) |
| 4033, 2 | $63 \cdot 2^6$ | 3521, **4032** ($= -1$) | "prime" (wrong: $37 \times 109$) |
| 4033, 3 | $63 \cdot 2^6$ | 3551, 2443, 3442, 2443, 3442, 2443 | composite (correct) |
| 2047, 2 | $1023 \cdot 2^1$ | **1** | "prime" (wrong: $M_{11} = 23 \times 89$) |
| 25, 7 | $3 \cdot 2^3$ | 18, **24** ($= -1$) | "prime" (wrong: $5^2$) |

Bases 2 for 4033 and 2047, and 7 for 25, are strong liars; this is exactly why the test is repeated with several bases (4033 fools base 2 but not base 3).

### 1.4 Factorization

**Fundamental theorem of arithmetic.** Every integer $n > 1$ can be written as a product of primes, $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$, and this representation is unique apart from the order of the factors. Uniqueness lets us read gcd and lcm

directly from the exponents. A common divisor can use each prime at most as often as *both* numbers contain it, and a common multiple must use it at least as often as *either* does:

$$\gcd(a,b) = \prod p_i^{\min(a_i,b_i)}, \qquad \operatorname{lcm}(a,b) = \prod p_i^{\max(a_i,b_i)}, \qquad \gcd(a,b) \times \operatorname{lcm}(a,b) = a \times b,$$

the last because $\min(a_i,b_i) + \max(a_i,b_i) = a_i + b_i$ for every prime. Example: $2160 = 2^4 \cdot 3^3 \cdot 5$ and $1200 = 2^4 \cdot 3 \cdot 5^2$ give gcd $= 2^4 \cdot 3 \cdot 5 = 240$ and lcm $= 2^4 \cdot 3^3 \cdot 5^2 = 10800$; check $240 \times 10800 = 2{,}592{,}000 = 2160 \times 1200$. This method is useless for large numbers, since it needs the factorization, which is exactly the hard part; Euclid's algorithm finds the gcd without it.

**Why factorization matters.** Finding $\phi(n)$, and with it the RSA private key, is as hard as factoring $n = pq$. Primality testing is easy (Miller–Rabin), but finding the factors of a number already known to be composite is believed to be hard, and RSA lives in the gap between the two problems.

**Trial division.** Divide $n$ by $a = 2, 3, 4, \ldots$ while $a \le \sqrt{n}$; whenever $a \mid n$, output $a$, replace $n$ by $n/a$, and try the same $a$ again (to catch repeated factors). If $n > 1$ when the loop ends, the remaining $n$ is a prime factor. Examples: $1233 = 3^2 \times 137$, and $1523357784 = 2^3 \times 3^2 \times 13 \times 37 \times 43987$. Like the divisibility test, it costs up to $\sqrt{n} = 2^{n_b/2}$ divisions, so it only works when $n$ has small factors.

**Fermat's factorization method.** Any odd $n = ab$ (with $a \ge b$) is a difference of two squares: with $x = (a+b)/2$ and $y = (a-b)/2$ we get $x^2 - y^2 = ab = n$. Conversely, if we find $x$ with $w = x^2 - n$ a perfect square $y^2$, then $n = (x+y)(x-y)$. The method starts at $x = \lceil\sqrt{n}\rceil$ and increases $x$ by 1 until $x^2 - n$ is a square. Since the search ends at $x = (a+b)/2$, it is fast exactly when the two factors are close to $\sqrt{n}$.

| $x$ | $w = x^2 - 5959$ | Square? | |
|---|---|---|---|
| 78 | 125 | no | $(\lceil\sqrt{5959}\rceil = 78)$ |
| 79 | 282 | no | |
| 80 | 441 | yes, $21^2$ | $5959 = (80+21)(80-21) =$ **101** × **59** |

For 8051 it succeeds immediately ($90^2 - 8051 = 49 = 7^2$, so $8051 = 97 \times 83$), while $4033 = 37 \times 109$ needs 10 steps ($x = 64, \ldots, 73$; $73^2 - 4033 = 1296 = 36^2$). The security lesson is that RSA primes $p$ and $q$ must not be close to each other.

**Other methods** (named on the slides). *Pollard* $p-1$ finds a prime factor $p$ quickly when $p-1$ has only small prime factors, by computing $\gcd(a^{B!} - 1, n)$. *Pollard rho* iterates $x \leftarrow x^2 + c \bmod n$ and finds a factor $p$ in about $\sqrt{p}$ steps from a repetition modulo $p$, detected with a gcd. The *quadratic sieve* and the *number field sieve* build many congruences $x^2 \equiv y^2 \pmod n$ (a generalisation of Fermat's idea) and are sub-exponential; the number field sieve is the fastest known method for large RSA moduli, but it is still far too slow for 2048-bit $n$. (A *sieve of Eratosthenes*, by contrast, is the school method for *listing* all primes up to a bound by crossing out multiples.)

### 1.5 Chinese remainder theorem ▸

▸ **Likely this year:** Forouzan style "Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$" with the full $M$, $M_i$, $M_i^{-1}$ table and the inverses by extended Euclid; or a word problem / RSA speed-up.

**Problem.** We are given several congruences in one unknown with different moduli,

$$x \equiv a_1 \pmod{m_1}, \quad x \equiv a_2 \pmod{m_2}, \quad \ldots, \quad x \equiv a_k \pmod{m_k},$$

and want all $x$ that satisfy them simultaneously. The Chinese remainder theorem (CRT) states that if the moduli are pairwise coprime, the system has exactly one solution modulo $M = m_1 m_2 \cdots m_k$, and it gives a formula for it. For example $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$ has the solution $x = 23$ (and $23 + 105t$).

**Idea.** Build $x$ as a sum of $k$ terms in which the $i$-th term is $\equiv a_i$ modulo $m_i$ and $\equiv 0$ modulo every other modulus. Then modulo $m_i$ all terms but the $i$-th vanish and $x \equiv a_i$, for each $i$ at once. A number divisible by every modulus except $m_i$ is $M_i = M/m_i$; multiplying it by its inverse modulo $m_i$ turns it into 1 modulo $m_i$ without disturbing the zeros, and multiplying by $a_i$ gives the required value.

**★ MUST KNOW : CRT in four steps**

1. $M = m_1 \times m_2 \times \cdots \times m_k$ (the common modulus). 2. $M_i = M/m_i$ for each $i$.
3. $M_i^{-1}$ = the inverse of $M_i$ **modulo** $m_i$ (reduce $M_i$ mod $m_i$ first, then extended Euclid).
4. $x = \left(a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + \cdots + a_k M_k M_k^{-1}\right) \bmod M$.

**Why it works.** For $j \ne i$, $m_j$ divides $M_i$, so the term $a_i M_i M_i^{-1}$ is $\equiv 0 \pmod{m_j}$. Modulo $m_i$ the term equals $a_i \cdot (M_i M_i^{-1}) \equiv a_i \cdot 1$. Therefore $x \equiv a_i \pmod{m_i}$ for every $i$. Step 3 needs $\gcd(M_i, m_i) = 1$, and this is exactly where pairwise coprimality is used: $M_i$ is a product of moduli each coprime to $m_i$.

**Why the solution is unique modulo $M$.** If $x$ and $x'$ both solve the system, every $m_i$ divides $x - x'$; since the $m_i$ are pairwise coprime, their product $M$ divides $x - x'$. So all solutions are $x + tM$, and there is exactly one in $\{0, \dots, M-1\}$. Equivalently, the map $x \mapsto (x \bmod m_1, \dots, x \bmod m_k)$ is a one-to-one correspondence between $\mathbb{Z}_M$ and $\mathbb{Z}_{m_1} \times \cdots \times \mathbb{Z}_{m_k}$, which is what the applications below exploit.

**Edge case: moduli not coprime.** Then the theorem does not apply, and the system may have no solution or several. For instance $x \equiv 1 \pmod 4$ and $x \equiv 2 \pmod 6$ has no solution, because the first forces $x$ odd and the second forces $x$ even.

**WORKED EXAMPLE : the standard CRT question (Forouzan, slide example)**

**Q.** Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$ using the Chinese remainder theorem.
**Step 1.** The moduli 3, 5, 7 are pairwise coprime, so the CRT applies. $M = 3 \times 5 \times 7 = 105$.
**Steps 2–3.**

| $i$ | $a_i$ | $m_i$ | $M_i = M/m_i$ | $M_i \bmod m_i$ | $M_i^{-1} \bmod m_i$ | $a_i M_i M_i^{-1}$ |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 35 | 2 | 2 | $2 \cdot 35 \cdot 2 = 140$ |
| 2 | 3 | 5 | 21 | 1 | 1 | $3 \cdot 21 \cdot 1 = 63$ |
| 3 | 2 | 7 | 15 | 1 | 1 | $2 \cdot 15 \cdot 1 = 30$ |

Inverses (extended Euclid with $r_1 = m_i$, $r_2 = M_i \bmod m_i$, $t_1 = 0$, $t_2 = 1$). For $35^{-1} \bmod 3 = 2^{-1} \bmod 3$:

| $q$ | $r_1$ | $r_2$ | $r$ | $t_1$ | $t_2$ | $t$ |
|---|---|---|---|---|---|---|
| 1 | 3 | 2 | 1 | 0 | 1 | −1 |
| 2 | 2 | 1 | 0 | 1 | −1 | 3 |
| | **1** | 0 | | **−1** | 3 | |

$r_1 = 1$, so the inverse exists and equals $t_1 = -1 \equiv \mathbf{2} \pmod 3$ (check $2 \cdot 2 = 4 \equiv 1$). Since $21 \equiv 1 \pmod 5$ and $15 \equiv 1 \pmod 7$, the other two inverses are 1.
**Step 4.** $x = (140 + 63 + 30) \bmod 105 = 233 \bmod 105 = \mathbf{23}$.
**Check.** $23 = 7 \cdot 3 + 2$, $23 = 4 \cdot 5 + 3$, $23 = 3 \cdot 7 + 2$ ✓. All solutions: $x = 23 + 105t$.

**WORKED EXAMPLE : CRT with non-trivial inverses (new)**

**Q.** Find the smallest positive $x$ with $x \equiv 3 \pmod 7$, $x \equiv 5 \pmod{11}$, $x \equiv 7 \pmod{13}$.
**Step 1.** 7, 11, 13 are distinct primes, hence pairwise coprime. $M = 7 \times 11 \times 13 = 1001$.
**Steps 2–3.**

| $i$ | $a_i$ | $m_i$ | $M_i$ | $M_i \bmod m_i$ | $M_i^{-1} \bmod m_i$ | $a_i M_i M_i^{-1}$ |
|---|---|---|---|---|---|---|
| 1 | 3 | 7 | 143 | 3 | 5 | $3 \cdot 143 \cdot 5 = 2145$ |
| 2 | 5 | 11 | 91 | 3 | 4 | $5 \cdot 91 \cdot 4 = 1820$ |
| 3 | 7 | 13 | 77 | 12 | 12 | $7 \cdot 77 \cdot 12 = 6468$ |

Inverses by extended Euclid (columns $q, r_1, r_2, r, t_1, t_2, t$):

$3^{-1} \bmod 7$

| $q$ | $r_1$ | $r_2$ | $r$ | $t_1$ | $t_2$ | $t$ |
|---|---|---|---|---|---|---|
| 2 | 7 | 3 | 1 | 0 | 1 | −2 |
| 3 | 3 | 1 | 0 | 1 | −2 | 7 |
| | **1** | 0 | | **−2** | 7 | |

$3^{-1} \bmod 11$

| $q$ | $r_1$ | $r_2$ | $r$ | $t_1$ | $t_2$ | $t$ |
|---|---|---|---|---|---|---|
| 3 | 11 | 3 | 2 | 0 | 1 | −3 |
| 1 | 3 | 2 | 1 | 1 | −3 | 4 |
| 2 | 2 | 1 | 0 | −3 | 4 | −11 |
| | **1** | 0 | | **4** | −11 | |

So $143^{-1} \equiv 3^{-1} \equiv -2 \equiv \mathbf{5} \pmod 7$ (check $3 \cdot 5 = 15 \equiv 1$) and $91^{-1} \equiv 3^{-1} \equiv \mathbf{4} \pmod{11}$ (check $3 \cdot 4 = 12 \equiv 1$). For the third, $77 \equiv 12 \equiv -1 \pmod{13}$, and $(-1)^{-1} = -1$, so $77^{-1} \equiv \mathbf{12} \pmod{13}$.
**Step 4.** $x = (2145 + 1820 + 6468) \bmod 1001 = 10433 \bmod 1001 = \mathbf{423}$.
**Check.** $423 = 60 \cdot 7 + 3$, $423 = 38 \cdot 11 + 5$, $423 = 32 \cdot 13 + 7$ ✓.

**Trap: inverses modulo the wrong number**

$M_i^{-1}$ is the inverse modulo $m_i$ (the small modulus), not modulo $M$. Reduce $M_i$ modulo $m_i$ before running extended Euclid, but multiply the *unreduced* $M_i$ in step 4; using $M_i \bmod m_i$ in the final sum breaks the zeros modulo the other moduli.

**More slide examples** (all checked):

| System | $M$ | $M_i$ | $M_i^{-1}$ | $x$ |
|---|---|---|---|---|
| $x \equiv 3\ (7),\ 3\ (13),\ 0\ (12)$ | 1092 | 156, 84, 91 | 4, 11, 7 | **276** |
| $S \equiv 1\ (11),\ 12\ (13),\ 2\ (17)$ | 2431 | 221, 187, 143 | 1, 8, 5 | **155** |
| $z \equiv 61\ (99),\ 65\ (98),\ 69\ (97)$ | 941094 | 9506, 9603, 9702 | 50, 97, 49 | **457** |

**Applications.**

- *Word problems.* "Find an integer that leaves remainder 3 when divided by 7 and by 13 but is divisible by 12" is the first row above: $x = 276$ ($276 = 12 \times 23$, $276 = 39 \cdot 7 + 3 = 21 \cdot 13 + 3$).
- *Secret sharing.* The secret $S$ is handed out as residues modulo 11, 13, 17, 19: shares 1, 12, 2, 3. Any 3 shares determine $S$ uniquely modulo the product of their moduli, which exceeds $S$, so CRT recovers it; every one of the four 3-subsets gives $S = 155$ (the second row uses $\{1, 12, 2\}$).
- *Arithmetic on big numbers with small registers.* To compute $z = 123 + 334$ on a machine that holds only numbers below 100, represent $123 \equiv 24, 25, 26$ and $334 \equiv 37, 40, 43$ modulo $99, 98, 97$; add componentwise to get $z \equiv 61, 65, 69$; CRT gives $z = 457$, correct because $457 < M = 941094$ and the solution is unique modulo $M$.
- *RSA decryption speed-up.* Since $n = pq$ with $p, q$ coprime, the plaintext $P = C^d \bmod n$ is determined by $P \bmod p$ and $P \bmod q$. These are computed with exponents reduced by Fermat, $d_p = d \bmod (p-1)$ and $d_q = d \bmod (q-1)$, on numbers half as long, and are then combined by CRT. A modular multiplication on half-length numbers costs about a quarter as much and the exponents are half as long, so the two half-size powers together are roughly four times faster than one full-size power.

**WORKED EXAMPLE : RSA decryption with CRT (new)**

**Q.** In RSA with $p = 7$, $q = 11$, $n = 77$, $e = 13$, $d = 37$, decrypt $C = 41$ using the CRT.
**Reduced exponents.** $d_p = 37 \bmod 6 = 1$, $d_q = 37 \bmod 10 = 7$.
**Half-size powers.** $P_p = 41^1 \bmod 7 = 6$.
$P_q = 41^7 \bmod 11 = 8^7 \bmod 11$: $8^2 = 64 \equiv 9$, $8^4 \equiv 81 \equiv 4$, $8^7 = 8^4 \cdot 8^2 \cdot 8 \equiv 4 \cdot 9 \cdot 8 = 288 \equiv 2$.
**CRT.** Solve $P \equiv 6 \pmod 7$, $P \equiv 2 \pmod{11}$ with $M = 77$:
$M_1 = 11 \equiv 4 \pmod 7$, $4^{-1} \bmod 7 = 2$; $M_2 = 7$, $7^{-1} \bmod 11 = 8$ ($7 \cdot 8 = 56 = 5 \cdot 11 + 1$).
$P = (6 \cdot 11 \cdot 2 + 2 \cdot 7 \cdot 8) \bmod 77 = (132 + 112) \bmod 77 = 244 \bmod 77 = \mathbf{13}$.
**Check.** Directly, $41^{37} \bmod 77 = 13$, and re-encrypting, $13^{13} \bmod 77 = 41$ ✓.

### 1.6 Fast exponentiation: square-and-multiply ★

▸ **Likely this year:** Needed inside every RSA / ElGamal / Diffie–Hellman numerical: show the bit table for $a^x \bmod n$.

**Problem.** Public-key cryptography computes $y = a^x \bmod n$ with $x$ and $n$ of 1024 bits or more. Multiplying by $a$ again and again needs $x - 1$ multiplications, about $2^{1024}$, which is impossible. (The size of intermediate results is the smaller problem: reducing modulo $n$ after every multiplication keeps them below $n^2$.)

**Idea.** Write the exponent in binary, $x = x_{n_b-1}2^{n_b-1} + \cdots + x_1 2^1 + x_0 2^0$ with each $x_i \in \{0, 1\}$. Then

$$a^x = \prod_{i\,:\,x_i=1} a^{2^i}, \qquad \text{e.g.} \quad a^{22} = a^{16} \cdot a^4 \cdot a^2 \quad (22 = 10110_2).$$

The powers $a, a^2, a^4, a^8, \ldots$ are each the square of the previous one, so they cost one squaring each, and we multiply into the result only those whose bit is 1.

**Mechanism (Forouzan, right-to-left).** Start with $y = 1$. Scan the bits from $x_0$ (least significant) upwards: if $x_i = 1$, set $y \leftarrow a \times y \bmod n$; then square, $a \leftarrow a^2 \bmod n$ (not needed after the last bit). Return $y$. An $n_b$-bit exponent needs at most $n_b - 1$ squarings and $n_b$ multiplications, so the cost is $O(n_b)$ modular multiplications, linear in the number of bits instead of exponential. A 1024-bit exponent needs at most about 2048 multiplications. The public exponent $e = 65537 = 10000000000000001_2$ needs 16 squarings and a single non-trivial multiplication, which is why it is the usual choice.

$17^{22} \bmod 21$ **(slide example),** $22 = 10110_2$

| $i$ | $x_i$ | Multiplication (init. $y = 1$) | Squaring (init. $a = 17$) |
|---|---|---|---|
| 0 | 0 | – | $a = 17^2 \bmod 21 = 16$ |
| 1 | 1 | $y = 1 \times 16 \bmod 21 = 16$ | $a = 16^2 \bmod 21 = 4$ |
| 2 | 1 | $y = 16 \times 4 \bmod 21 = 1$ | $a = 4^2 \bmod 21 = 16$ |
| 3 | 0 | – | $a = 16^2 \bmod 21 = 4$ |
| 4 | 1 | $y = 1 \times 4 \bmod 21 = \mathbf{4}$ | (not needed) |

So $17^{22} \bmod 21 = 4$, using 4 squarings and 3 multiplications instead of 21 multiplications. Section **??** computes the same kind of power left-to-right (scanning from the most significant bit, square then multiply by $a$); both give the same answer, and either is acceptable in the exam as long as every intermediate value is shown.

**WORKED EXAMPLE : RSA encryption and decryption by square-and-multiply (Forouzan numbers)**

**Q.** In RSA, $n = 77$, $e = 13$, $d = 37$. Encrypt $M = 5$ and verify by decrypting, using the square-and-multiply method.
**Encryption** $C = 5^{13} \bmod 77$, $13 = 1101_2$ ($x_0 = 1, x_1 = 0, x_2 = 1, x_3 = 1$):

| $i$ | $x_i$ | Multiplication ($y = 1$) | Squaring ($a = 5$) |
|---|---|---|---|
| 0 | 1 | $y = 1 \times 5 = 5$ | $a = 5^2 = 25$ |
| 1 | 0 | – | $a = 25^2 = 625 \bmod 77 = 9$ |
| 2 | 1 | $y = 5 \times 9 = 45$ | $a = 9^2 = 81 \bmod 77 = 4$ |
| 3 | 1 | $y = 45 \times 4 = 180 \bmod 77 =$ **26** | (not needed) |

**Decryption** $M = 26^{37} \bmod 77$, $37 = 100101_2$:

| $i$ | $x_i$ | Multiplication ($y = 1$) | Squaring ($a = 26$) |
|---|---|---|---|
| 0 | 1 | $y = 26$ | $a = 26^2 = 676 \bmod 77 = 60$ |
| 1 | 0 | – | $a = 60^2 = 3600 \bmod 77 = 58$ |
| 2 | 1 | $y = 26 \times 58 = 1508 \bmod 77 = 45$ | $a = 58^2 = 3364 \bmod 77 = 53$ |
| 3 | 0 | – | $a = 53^2 = 2809 \bmod 77 = 37$ |
| 4 | 0 | – | $a = 37^2 = 1369 \bmod 77 = 60$ |
| 5 | 1 | $y = 45 \times 60 = 2700 \bmod 77 =$ **5** | (not needed) |

**Answer.** $C = 26$, and decryption returns $M = 5$ ✓.

### 1.7 Discrete logarithms ▸

▸ **Likely this year:** "Find $x$ such that $3^x \equiv 7 \pmod{17}$" via a table of powers, using the log rules; or theory: why the discrete log is hard and which systems rely on it.

**Problem.** Square-and-multiply computes $y = a^x \bmod n$ quickly. The reverse question is: given $a$, $y$ and $n$, find $x$. Over the real numbers this is $\log_a y$; modulo $n$ it is the *discrete logarithm*, and no comparably fast method is known for it.

**Primitive roots (reminder).** The order of $a$ in $\mathbb{Z}_n^*$ is the smallest $i \geq 1$ with $a^i \equiv 1$, and $a$ is a *primitive root* when its order equals $\phi(n)$; details and examples are in §**??**. For a prime $p$, $\mathbb{Z}_p^*$ always has primitive roots, and if $g$ is one then $g^1, g^2, \ldots, g^{p-1}$ are all different and run through every element $1, \ldots, p-1$ exactly once. Hence every $y \in \mathbb{Z}_p^*$ equals $g^x$ for exactly one $x \in \{1, \ldots, p-1\}$; this $x$ is the *discrete logarithm* of $y$ to the base $g$, written $x = L_g\, y$ (the modulus is understood). In $\mathbb{Z}_7^*$ the primitive roots are 3 and 5, giving two logarithm tables:

| $y$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $x = L_3\, y \pmod 7$ | 6 | 2 | 1 | 4 | 5 | 3 |
| $x = L_5\, y \pmod 7$ | 6 | 4 | 5 | 2 | 1 | 3 |

With the table, $4 \equiv 3^x \pmod 7$ gives $x = L_3 4 = 4$ ($3^4 = 81 \equiv 4$), and $6 \equiv 5^x \pmod 7$ gives $x = L_5 6 = 3$ ($5^3 = 125 \equiv 6$).

**Rules.** Since $g^{p-1} \equiv 1$, exponents of $g$ only matter modulo $p - 1 = \phi(p)$, so discrete logs obey the usual logarithm laws with arithmetic modulo $\phi(n)$:

$$L_g 1 \equiv 0, \qquad L_g(x \cdot y) \equiv L_g x + L_g y, \qquad L_g(x^k) \equiv k \cdot L_g x \pmod{\phi(n)}.$$

The second rule follows from $g^{Lx} \cdot g^{Ly} = g^{Lx+Ly}$. (The table writes $L_g 1 = 6$ rather than 0, because it lists exponents from 1 to $p - 1$; $6 \equiv 0 \pmod 6$.) Example in $\mathbb{Z}_7^*$: $L_3(4 \cdot 5) = L_3 6 = 3$, and $L_3 4 + L_3 5 = 4 + 5 = 9 \equiv 3 \pmod 6$ ✓.

**Edge case: base not a primitive root.** Then the powers of $a$ do not cover the whole group, so $a^x \equiv y$ may have no solution or several. In $\mathbb{Z}_7^*$, 2 has order 3 ($2^x$ cycles through 2, 4, 1), so $2^x \equiv 3 \pmod 7$ has no solution, while $2^x \equiv 4$ has $x = 2$ and $x = 5$.

**Why it is hard.** The only general method on the slides is *exhaustive search*: for $x = 1, 2, \ldots, n-1$, test whether $y = a^x \bmod n$ and return the first $x$ that works. That is up to $p - 1$ exponentiations, exponential in the number of bits, while computing $a^x$ needs only $O(\log p)$ multiplications. Better algorithms exist (baby-step giant-step in about $\sqrt{p}$ steps; index calculus, sub-exponential), but for a well-chosen 2048-bit prime they are still infeasible, roughly as hard as factoring a 2048-bit RSA modulus. Modular exponentiation is therefore a *one-way function*, and this asymmetry is the basis of **Diffie–Hellman** (both parties publish $g^a$ and $g^b$ and share $g^{ab}$; an eavesdropper would need $a = L_g(g^a)$) and of **ElGamal** (the private key $d$ hides behind the public $e_2 = e_1^d \bmod p$). For example, with $p = 17$, $g = 3$, $a = 4$, $b = 7$: the public values are $3^4 \equiv 13$ and $3^7 \equiv 11$, and the shared key is $11^4 \equiv 13^7 \equiv 4 \pmod{17}$.

**WORKED EXAMPLE : discrete logarithms modulo 17 (new)**

**Q.** (a) Show that 3 is a primitive root of 17 and tabulate $L_3\,y$ for $\mathbb{Z}_{17}^{\star}$. (b) Solve $3^x \equiv 7 \pmod{17}$. (c) Using only $L_3 2$ and $L_3 7$ and the rules, find $L_3 14$. (d) Solve $3^x \equiv 8 \pmod{17}$ from $L_3 2$.

**(a)** Successive powers $3^x \bmod 17$, each 3× the previous:

| $x$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $3^x \bmod 17$ | 3 | 9 | 10 | 13 | 5 | 15 | 11 | 16 | 14 | 8 | 7 | 4 | 12 | 2 | 6 | 1 |

The first power equal to 1 is $3^{16}$, so $\operatorname{ord}(3) = 16 = \phi(17)$ and 3 is a primitive root (all 16 values differ). Reading the table backwards:

| $y$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $L_3\,y$ | 16 | 14 | 1 | 12 | 5 | 15 | 11 | 10 | 2 | 3 | 7 | 13 | 4 | 9 | 6 | 8 |

**(b)** $x = L_3 7 =$ **11**. Check: $3^{11} \bmod 17 = 7$ (from the power table).
**(c)** $L_3 14 = L_3(2 \cdot 7) \equiv L_3 2 + L_3 7 = 14 + 11 = 25 \equiv$ **9** $\pmod{16}$. Check: $3^9 \bmod 17 = 14$ ✓.
**(d)** $L_3 8 = L_3(2^3) \equiv 3 \cdot L_3 2 = 42 \equiv$ **10** $\pmod{16}$. Check: $3^{10} \bmod 17 = 8$ ✓.

**Corrections to the notes / slides**

- Slide (Fermat test): "561 = 33 × 17" is true but is not the prime factorisation; 561 = 3 × 11 × 17, the smallest Carmichael number.
- Slide (AKS): the example computes $(\log_2 200)^{12} \approx 3.9 \times 10^{10}$ bit operations for a 200-bit $n$ (the printed 39,547,615,483 is not even that value exactly; it is ≈ 39,787,815,242). For a 200-bit number $\log_2 n = 200$, so the bound $(\log_2 n)^{12}$ is $200^{12} \approx 4.1 \times 10^{27}$: still polynomial, but not "40 seconds".
- Slide (recommended test, 4033): the list of trial divisors "2, 3, 5, 7, 11, 17, 23" skips 13 and 19 (neither divides 4033). A full divisibility test up to $\lfloor\sqrt{4033}\rfloor = 63$ would find 37 and settle the question; the recommended test uses trial division by *small* primes only.
- Slide (Fermat test): "If $x$ is a prime, then $a^{n-1} \equiv 1$" should read "if $n$ is a prime".
- Slide $\phi(1) = 0$ follows Forouzan's "integers smaller than $n$"; the standard convention is $\phi(1) = 1$. It never matters in RSA. (Totient chapter.)
- Euler's second version $a^{k\phi(n)+1} \equiv a \pmod{n}$ holds for squarefree $n$ (such as $n = pq$ in RSA), not for every $n$; it fails for $n = 4$, $a = 2$. (See §**??**.)

**Check your understanding**

1. **Why is it enough to test divisors up to $\sqrt{n}$, and why is this still too slow for RSA?** If $n = ab$ with $a \le b$, then $a \le \sqrt{n}$, so a composite always has a factor at most $\sqrt{n}$. But $\sqrt{n} = 2^{n_b/2}$ for an $n_b$-bit number, which is exponential in the input size ($2^{512}$ trials for a 1024-bit number).
2. **Differentiate deterministic and probabilistic primality tests.** A deterministic test (divisibility, AKS) always answers correctly; a probabilistic test (Fermat, Miller–Rabin) is fast and its "composite" answer is always correct, but its "prime" answer may be wrong with a small probability that shrinks with every extra base tried ($\le 4^{-t}$ for Miller–Rabin).
3. **What is a Carmichael number, and why does 561 pass the Fermat test for every coprime base?** A composite $n$ with $a^{n-1} \equiv 1$ for all $a$ coprime to $n$. For $561 = 3 \cdot 11 \cdot 17$, each $p - 1 \in \{2, 10, 16\}$ divides 560, so $a^{560} \equiv 1$ modulo 3, 11 and 17, hence modulo 561 by the CRT.
4. **Apply Miller–Rabin with base 2 to 221.** $220 = 55 \cdot 2^2$; $T = 2^{55} \bmod 221 = 128$ (not ±1); one squaring: $128^2 \bmod 221 = 30$ (not ±1). The loop ends without a decision, so 221 is composite (= 13 × 17).
5. **Why must the CRT moduli be pairwise coprime?** Step 3 needs $M_i^{-1} \bmod m_i$, which exists only if $\gcd(M_i, m_i) = 1$, and uniqueness needs lcm of the moduli to equal their product. Without it the system can be unsolvable: $x \equiv 1 \pmod 4$, $x \equiv 2 \pmod 6$ has no solution.
6. **Solve** $x \equiv 1 \pmod 5$, $x \equiv 2 \pmod 7$, $x \equiv 3 \pmod 9$, $x \equiv 4 \pmod{11}$. $M = 3465$; $M_i = 693, 495, 385, 315$; reduced $3, 5, 7, 7$; inverses $2, 3, 4, 8$; $x = (1386 + 2970 + 4620 + 10080) \bmod 3465 = 19056 \bmod 3465 = 1731$.
7. **How many modular multiplications does $a^{65537} \bmod n$ need, and why?** $65537 = 2^{16} + 1$ has 17 bits with two 1s, so 16 squarings and one non-trivial multiplication (17 in total) instead of 65536.
8. **Why does Fermat factorization break 8051 at once, and what does that teach about RSA?** $\lceil\sqrt{8051}\rceil = 90$ and $90^2 - 8051 = 49 = 7^2$, so $8051 = 97 \times 83$. The method ends after about $(p + q)/2 - \sqrt{n}$ steps, so $p$ and $q$ must not be close together.
9. **Solve** $2^x \equiv 9 \pmod{11}$. 2 is a primitive root of 11 (powers $2, 4, 8, 5, 10, 9, \ldots$), and $2^6 = 64 \equiv 9$, so $x = 6$ (unique modulo 10).
10. **Why is the discrete logarithm a good basis for public-key cryptography?** Computing $g^x \bmod p$ takes $O(\log p)$ multiplications by square-and-multiply, but recovering $x$ needs exhaustive search over up to $p - 1$ exponents

(or still-infeasible sub-exponential algorithms for 2048-bit $p$). This one-way property protects the Diffie–Hellman shared key and the ElGamal private key.